

MediReach AI

Intelligent WhatsApp-Driven Healthcare Platform

Technical Architecture & Workflow Documentation

Version	1.0.0 — Initial Production Release
Status	Production Ready
Platform	n8n · Medplum FHIR · Google Gemini AI · WhatsApp Business API
Compliance	HL7 FHIR R4 · HIPAA-aligned Data Architecture · OAuth2 Authentication

Table of Contents

1. Executive Summary

1.1 Key Value Propositions

2. System Architecture Overview

2.1 Technology Stack

2.2 High-Level Data Flow

3. Workflow 1 — Clinic & Practitioner Onboarding

3.1 Purpose & Scope

3.2 Trigger

3.3 Node-by-Node Technical Breakdown

3.4 Design Decisions & Rationale

4. Workflow 2 — Core AI Clinical Agent

4.1 Purpose & Scope

4.2 Layer 1 — Message Ingestion

4.3 Layer 2 — Security & Spam Protection

4.4 Layer 3 — Media Type Routing

4.5 Layer 4 — Intent Classification

4.6 Layer 5 — AI Clinical Agent

4.7 Layer 6 — Long-Term Memory Pipeline

5. Workflow 3 — Document Upload Sub-Workflow

5.1 Trigger & Entry

5.2 Phase 1 — Patient Identification

5.3 Phase 2 — Media Type Detection

5.4 Phase 3 — Download & MIME Normalization

5.5 Phase 4 — 12-Route Document Classification

5.6 Phase 5 — Content Extraction

5.7 Phase 6 — Binary Upload Path

5.8 Error Handling

6. Workflow 4 — Smart Patient Departure Notifier

- 6.1 [Purpose](#)
- 6.2 [Node-by-Node Breakdown](#)
- 6.3 [Parallel Execution Architecture](#)

7. Security Architecture

- 7.1 [Authentication](#)
- 7.2 [Abuse Prevention](#)
- 7.3 [Data Privacy](#)

8. FHIR Resource Reference

9. End-to-End Patient Journey Example

10. Glossary

1. Executive Summary

MediReach AI is a production-grade, WhatsApp-native clinical intelligence platform that eliminates the friction between patients and healthcare providers. Built on an HL7 FHIR R4 compliant backend (Medplum), it orchestrates the entire patient journey — from clinic onboarding through appointment booking, medical record management, and real-time departure optimization — through a single familiar interface: WhatsApp.

Unlike traditional patient portals that require app downloads and lengthy registrations, MediReach AI meets patients exactly where they already are. A patient can book an appointment, report symptoms, receive medical guidance, and be told precisely when to leave home — all without leaving WhatsApp.

The platform is composed of four tightly integrated automation workflows, each serving a distinct operational layer:

- Clinic & Practitioner Onboarding — Automated FHIR Organization and Practitioner provisioning from a single JotForm submission.
- Core AI Clinical Agent — The conversational AI brain that handles patient identity, appointment management, symptom logging, and persistent medical memory.
- Document Ingestion Pipeline — A 12-route intelligent parser that processes patient-uploaded files and attaches them to FHIR records.
- Smart Departure Notifier — A real-time queue analysis and travel-time engine that tells patients exactly when to leave home.

1.1 Key Value Propositions

Capability	Business Impact
Zero-app patient interface	100% adoption with no patient onboarding friction — patients use the WhatsApp they already have.
FHIR R4 compliance	All patient data is structured, interoperable, and ready for health information exchange.
AI-driven triage	Gemini 2.5 Pro classifies intent and handles clinical conversations with sub-second latency.
Smart queue management	Patients receive personalized departure times, eliminating waiting-room overcrowding.
Persistent clinical memory	Semantic vector memory ensures the AI remembers patient history across sessions.
Multi-tenant architecture	A single deployment serves multiple clinics simultaneously via organization-scoped records.

2. System Architecture Overview

MediReach AI is built on a layered, event-driven architecture. At its foundation sits the Medplum FHIR platform, acting as the secure, structured healthcare data store. Above this, n8n automation workflows orchestrate all business logic, connecting WhatsApp as the patient-facing channel with Medplum, Google Gemini AI models, Redis, and Supabase as supporting infrastructure.

2.1 Technology Stack

Layer	Technology	Role
Automation Engine	n8n	Workflow orchestration, node execution, API coordination
Healthcare Backend	Medplum (FHIR R4)	Patient, Practitioner, Organization, Appointment, Slot, Observation, Binary, DocumentReference resources
AI / LLM	Google Gemini 2.5 Pro / Flash	Conversational agent, intent classification, audio transcription, memory extraction
Vector Memory	Supabase + Gemini Embeddings	Long-term semantic patient memory store
Session Memory	Redis	Short-term conversation history (20 turns, 1 hr TTL) and rate-limit counters
Patient Channel	WhatsApp Business API	Inbound message receipt; outbound notification delivery
Onboarding Form	JotForm	Clinic and practitioner registration data capture
Routing & Maps	OpenRouteService API	Patient address geocoding and driving-time calculation

2.2 High-Level Data Flow

The following describes the end-to-end journey of a patient interaction from message receipt to clinical record update:

1. Patient sends a WhatsApp message to the clinic's business number.
2. The WhatsApp Trigger node receives the webhook payload (text, audio, image, or document).
3. The security layer validates the sender against rate limits and the Supabase blocked-users table.
4. Media routing normalizes the message to plain text — transcribing audio via Gemini Flash if needed — or dispatches files to the Document Upload sub-workflow.
5. Intent classification categorizes the message as medical, greeting, spam, or irrelevant.
6. The AI Clinical Agent processes medical and greeting intents, calling FHIR tools as needed.
7. The agent's response is delivered back to the patient via WhatsApp.
8. In parallel, a background memory pipeline extracts clinical facts and stores them as vector embeddings in Supabase for future retrieval.

3. Workflow 1 — Clinic & Practitioner Onboarding

This workflow is the administrative entry point for all healthcare providers joining the MediReach AI platform. It automates the provisioning of FHIR-compliant Organization and Practitioner records in Medplum directly from a JotForm submission, with zero manual intervention.

3.1 Purpose & Scope

- Accepts a single form submission from a clinic administrator or individual practitioner.
- Automatically creates an Organization (clinic) record, a Practitioner (doctor/clinician) record, or both — depending on which fields were completed.
- Links both records together via a FHIR PractitionerRole resource, formally establishing the practitioner's affiliation with the clinic.
- Handles partial submissions gracefully — if only one entity is submitted, only that record is created.

3.2 Trigger

The workflow is activated by a JotForm webhook (Form ID: 260714900434047). When a form is submitted, JotForm immediately POSTs the entire payload as JSON to the n8n webhook endpoint, which fires the workflow.

3.3 Node-by-Node Technical Breakdown

Node 1 — JotForm Trigger

Entry point. Listens via webhook for new form submissions. The payload contains all raw form field values — organization details, practitioner details, and qualification data — delivered as a flat JSON object.

Node 2 — Edit Fields (Data Restructuring)

Performs a shape transformation on the raw JotForm output. No business logic — purely renaming and grouping. Restructures the flat form payload into two clean named objects:

- New Organization — containing: Type, Name, Alias, Telecom (phone), Address, Contact (email).
- New Practitioner — containing: Name, Telecom, Address, Gender, BirthDate, Qualification, Communication language.

Node 3 — Code (JS2): Data Sanitization

A JavaScript preprocessing node responsible for data integrity before any API calls are made. Key operations:

- BirthDate normalization: The birthDate field can arrive as a structured object {year, month, day} or as an empty placeholder string "/". This node converts a valid date object into YYYY-MM-DD format (FHIR-required) or sets it to null if empty or malformed.

- Null safety guards: Ensures the New Organization and New Practitioner objects always exist as empty objects rather than undefined, preventing null-reference crashes in downstream nodes.

Parallel Branch Split

After Node 3, the workflow splits into two independent, concurrent execution branches — Organization (left) and Practitioner (right). Both branches receive the same sanitized data and run simultaneously.

Node 4 — If (Organization Branch Guard)

Checks whether New Organization.Name is non-empty. If the clinic name was provided, the branch continues. If not, the entire Organization branch terminates here — preventing the creation of empty or invalid FHIR records.

Node 5 — Access Medplum Token (Organization)

Performs an OAuth2 client_credentials POST to the Medplum token endpoint (<https://api.medplum.com/oauth2/token>). On success, receives a short-lived Bearer access token used to authorize all subsequent Medplum API calls in this branch.

Node 6 — Create the Organization

POSTs a fully structured FHIR R4 Organization resource to <https://api.medplum.com/fhir/R4/Organization>. The resource payload includes:

- resourceType: "Organization"
- type: HL7 V3 healthcare organization code
- name and alias from form data
- telecom: phone number
- address: structured postal address
- contact: email address

Node 7 — Code (JS): Parse Organization Response

The Medplum API returns the created Organization resource with a system-generated id. This node parses the raw response string into a proper JSON object so the Organization id is accessible downstream for the PractitionerRole linkage.

Node 8 — If2 (Practitioner Branch Guard)

Identical logic to Node 4, applied to the Practitioner branch. Checks whether New Practitioner.Name is non-empty before allowing the branch to continue.

Node 9 — Access Medplum Token1 (Practitioner)

An independent token fetch for the Practitioner branch. Because parallel branches in n8n are self-contained execution paths, each branch fetches its own authentication token rather than sharing one — avoiding token-expiry or race-condition issues.

Node 10 — Create The Practitioner

The most complex node in the workflow. POSTs a FHIR R4 Practitioner resource to Medplum with the following inline transformations:

- Name splitting: Splits the full name into a prefix array using `split(" ")` then `JSON.stringify` for FHIR HumanName format.
- BirthDate formatting: Converts MM/DD/YYYY to FHIR-required YYYY-MM-DD. Omits the field entirely if empty or malformed.
- Gender normalization: Lowercases the gender value to match FHIR's required enum (male / female / unknown).
- Qualification mapping: Maps human-readable degree names to HL7 qualification codes — e.g., "Associate of Applied Science" → AAS, "Doctor of Medicine" → MD.
- Communication language mapping: Maps "English" → en, "Arabic" → ar, with Urdu (ur) as the default fallback.

Node 11 — Code (JS1): Parse Practitioner Response

Parses the Medplum Practitioner creation response to extract the system-generated Practitioner id, making it available for the PractitionerRole linking step.

Node 12 — Merge

A synchronization point that waits for both the Organization and Practitioner branches to complete before proceeding. Uses "combine all" merge mode to produce a single output item containing both branch results — and therefore both generated IDs.

Node 13 — If1 (Dual-ID Safety Check)

A final guard before PractitionerRole creation. Verifies that both the Organization ID and Practitioner ID are non-empty. If either record creation failed upstream, this node prevents the linking step from executing with partial or null data.

Node 14 — PractitionerRole (FHIR Linkage)

The concluding step. POSTs a FHIR PractitionerRole resource to Medplum that formally associates the Practitioner with the Organization:

- organization: Reference to Organization/{org_id}
- practitioner: Reference to Practitioner/{practitioner_id}

This is the FHIR-standard method for expressing an employment or affiliation relationship between a practitioner and a healthcare organization.

3.4 Design Decisions & Rationale

Design Decision	Rationale
Parallel branching for Org & Practitioner	Reduces total provisioning time — both FHIR records are created concurrently rather than sequentially.
Independent token fetch per branch	Avoids token-sharing race conditions in parallel execution paths. Each branch is fully self-contained.
If-guards on both branches	Ensures the workflow handles partial form submissions gracefully. A clinic-only or practitioner-only submission is valid and processed correctly.

PractitionerRole only after both IDs confirmed	Prevents orphaned or broken FHIR relationships if either creation step fails.
--	---

4. Workflow 2 — Core AI Clinical Agent

This is the primary patient-facing workflow and the operational heart of MediReach AI. It receives every inbound WhatsApp message, applies security validation, routes by media type, classifies intent, and passes clinical or conversational messages to a fully-equipped AI agent with direct access to 13 FHIR tools in real time.

4.1 Purpose & Scope

- Serves as the 24/7 patient-facing interface for the clinic via WhatsApp.
- Protects the system from abuse with rate limiting and permanent blocking mechanisms.
- Handles text, audio, image, and document message types.
- Classifies patient intent to route messages efficiently and prevent LLM misuse.
- Executes clinical workflows (appointment booking, symptom logging, patient lookup) through a structured AI agent with direct FHIR access.
- Maintains conversational context via Redis (short-term) and clinical memory via Supabase vector store (long-term).

4.2 Layer 1 — Message Ingestion

WhatsApp Trigger

The entry point for all patient interactions. A webhook listener connected to the clinic's WhatsApp Business account. Fires on every inbound message regardless of type — text, audio, image, or document. Captures the sender's phone number, display name, and raw message content for all downstream processing.

4.3 Layer 2 — Security & Spam Protection

The first processing layer after message receipt. Two parallel security checks run simultaneously before any clinical logic executes:

Rate Limit Track

- `check_rate_limit`: Increments a Redis counter keyed to the sender's phone number. Tracks message frequency per time window.
- `validate_limit`: Checks if the counter exceeds 15 messages per window. If over threshold, branches to the spam escalation path.
- `check_spam_count`: Increments a secondary Redis counter for confirmed spam events from this number.
- `validate_spam_count`: If spam events exceed 5, triggers permanent blocking.
- `block_user`: Writes the phone number to the Supabase `blocked_users` table as a permanent record.
- An appropriate WhatsApp warning or block notification is sent and execution halts.

Blocklist Check Track

- `Get blocked users`: Queries the Supabase `blocked_users` table directly for the sender's phone number.

- isUserBlock: If a matching record is found, a block message is sent and execution halts.
- If not blocked, the path flows to a Merge node that reunites both security tracks before proceeding to message processing.

4.4 Layer 3 — Media Type Routing

Route Types is a Switch node that inspects the message type field and routes to one of four processing branches:

Message Type	Processing Path
Text	Message body extracted directly by Map text prompt into a clean message variable.
Audio	Gets WhatsApp Voicemail Source URL → Download Voicemail fetches binary audio → Transcribe via Gemini 2.5 Flash converts to text → Map text prompt 2 normalizes to the same message variable.
Image	Dispatched to the Document Upload Sub-Workflow for FHIR storage.
Document	Dispatched to the Document Upload Sub-Workflow for FHIR storage.

After this layer, all text and audio messages present as identical normalized message variables. The media routing is completely transparent to the AI agent.

4.5 Layer 4 — Intent Classification

Analyze the Intent

Sends the normalized message text to Gemini 2.5 Flash with a structured classification prompt. Returns one of four intent categories:

- medical — Clinical queries, symptom reports, appointment requests, prescription questions.
- greeting — Salutations, small talk, acknowledgements.
- spam — Nonsensical, repetitive, or clearly abusive content.
- irrelevant — Off-topic content unrelated to healthcare.

Code in JavaScript (Intent Sanitizer)

Post-processes the LLM classification output. Strips quotes, lowercases the result, and falls back to "irrelevant" if the model returns anything unexpected — ensuring the intent router always receives a valid, predictable value.

Intent Router (Switch Node)

- greeting or medical → routes to the AI Agent for full clinical processing.
- spam → sends a short WhatsApp warning message and terminates.
- irrelevant → sends a redirect message informing the patient the system is specialized for healthcare questions, and terminates.

4.6 Layer 5 — AI Clinical Agent

The AI Agent is the intelligence core of the platform. Implemented as an n8n LangChain agent powered by Google Gemini 2.5 Pro (temperature 0.1 for clinical precision), it receives the patient's message enriched with contextual data — patient name, WhatsApp number, and clinic phone — and can autonomously call any of the following 13 FHIR tools:

Identity & Patient Management Tools

Tool Name	Description
Get_Organization_context	Looks up the clinic's Organization record in Medplum using the incoming WhatsApp number. Enables multi-tenant support — the same agent serves multiple clinics by scoping all records to the correct organization.
medplum_patient_lookup	Searches Medplum for an existing patient by phone number. If no record exists, the agent collects name and gender and calls create_patient to register them.
get_patient_summary	Fetches the full Patient FHIR resource by UUID for identity confirmation and profile display.

Appointment Management Tools

Tool Name	Description
get_clinic_staff	Fetches PractitionerRole records to list available doctors for the patient to choose from.
get_schedule	Retrieves a doctor's Schedule FHIR resource by their Practitioner ID.
get_doctor_slots	Queries free Slot resources for a given schedule, filtering to future slots only, sorted by start time.
book_appointment	POSTs a new FHIR Appointment resource with references to the patient, practitioner, slot, and appointment time.
update_slot_status	PATCHes the booked Slot resource to 'busy' status, preventing double-booking.
cancel_or_reschedule_appointment	PATCHes an existing Appointment's status or slot reference for cancellations or simple reschedules.
get_appointment	GETs a specific Appointment resource to verify its current status before making changes.
reschedule_appointment	Calls a dedicated reschedule sub-workflow (ID: zQfINekkLeM1sDWU) for complex multi-step rescheduling scenarios.

Clinical Record Tools

Tool Name	Description
record_observation	POSTs a FHIR Observation using LOINC code 11488-4 (Consultation note) with the patient's symptom description linked to their record.

get_observation	Retrieves a patient's previous observations sorted by most recent, providing the agent with clinical history context.
-----------------	---

Memory Tool

Tool Name	Description
retrieve_patient_memory	Performs semantic vector search against the Supabase patient_memory table using Gemini embeddings, filtered by patient phone number. Returns the most contextually relevant past clinical facts for the current conversation.

Supporting Infrastructure

- Google Gemini Chat Model (Gemini 2.5 Pro): The LLM backend powering the agent's reasoning and responses.
- Redis Chat Memory: Stores the last 20 conversation turns per patient (keyed by phone number, 1-hour TTL), providing short-term conversational context for coherent multi-turn interactions.
- Date & Time Tool: Provides the agent with the current datetime in the Asia/Kolkata timezone for accurate appointment scheduling.
- Timezone conversion: The agent automatically converts all UTC slot times from Medplum to Pakistan Standard Time (UTC+5) before presenting them to patients.

Once the agent produces its response, the Send Message node delivers it back to the patient via the WhatsApp Business API.

4.7 Layer 6 — Long-Term Memory Pipeline

After every agent response, a parallel background pipeline extracts and stores clinically significant facts for future retrieval. This runs entirely asynchronously and does not delay the patient's response.

Memory Extraction

- Message a model (Gemma-3-1B): Analyzes the agent's response and extracts medically relevant facts. Returns a structured JSON object: { "save_required": true, "fact": "Patient reports headaches for 3 days" }.
- Code in JavaScript1: Parses the raw model output string into a structured JSON object.
- Filter: Checks whether save_required is true. If false — e.g., a greeting or scheduling confirmation — the pipeline stops, avoiding unnecessary storage.

Memory Storage

- Default Data Loader: Prepares the extracted fact text for chunking.
- Recursive Character Text Splitter: Splits text into semantic chunks (chunk size: 10 tokens).

- `save_patient_memory`: Inserts the chunk into Supabase's `patient_memory` vector store, generating embeddings using Google embedding-001. These vectors are available for future `retrieve_patient_memory` tool calls.

5. Workflow 3 — Document Upload Sub-Workflow

This sub-workflow is invoked by the Core AI Agent whenever a patient sends an image or document via WhatsApp. It identifies the patient, downloads the file, classifies its MIME type across 12 possible routes, extracts content where applicable, and either returns structured text to the parent workflow for AI processing or permanently attaches binary files to the patient's FHIR record in Medplum.

5.1 Trigger & Entry

Activated via n8n's sub-workflow execution mechanism — it does not run independently. The parent AI Agent workflow passes the full WhatsApp message payload as input. Simultaneously, the patient lookup begins and the raw message payload is forwarded to a Merge node for later combination with the patient identity data.

5.2 Phase 1 — Patient Identification

- `get_patient_id`: GETs `https://api.medplum.com/fhir/R4/Patient?telecom=+{wa_id}` to locate the patient's FHIR record by WhatsApp number.
- `Code in JavaScript2`: Parses the FHIR bundle response. Sets `found: true` and extracts `patient_id` if a record exists; sets `patient_id: null` and `found: false` if not.
- `If1`: Routes only confirmed patients downstream. Unknown phone numbers are silently dropped at this stage.
- `Merge`: Combines `patient_id` data with the original WhatsApp message payload, ensuring all downstream nodes have access to both.

5.3 Phase 2 — Media Type Detection

- `If2`: Checks whether `messages[0].type` equals "image". Routes to `Edit Fields1` (image path) or `Edit Fields2` (document path).
- `Edit Fields1 (Image)`: Extracts `messages[0].image.id` → `mediaId` and `messages[0].image.mime_type` → `mine_type`.
- `Edit Fields2 (Document)`: Extracts `messages[0].document.id` → `mediaId` and `messages[0].document.mime_type` → `mine_type`.
- `Edit Fields3 (Normalize)`: Convergence point for both branches. Ensures consistent field names (`mediaId`, `mine_type`) regardless of source media type.

5.4 Phase 3 — Download & MIME Normalization

- `Gets WhatsApp Document Source URL`: Calls the WhatsApp API to retrieve the temporary signed media URL for the given `mediaId`.
- `Download Document`: HTTP GET to the signed URL with WhatsApp OAuth2 authentication. Returns binary file data.
- `Map file extensions (Code node)`: Normalizes MIME types to internal aliases for consistent routing:
 - `text/calendar`, `application/ics`, `text/x-calendar` → `mapped/calendar`
 - `application/xml`, `text/xml` → `mapped/xml`
 - `image/jpeg`, `image/png`, `image/heic` → `mapped/image`

- Falls back to WhatsApp source URL metadata if binary MIME type is missing.

5.5 Phase 4 — 12-Route Document Classification

Route Document Types is a Switch node that inspects the normalized mime_type and dispatches to the appropriate processing path:

Route Key	MIME Type Matched	Destination
CSV	text/csv	Map document prompt
HTML	text/html	Map document prompt
Calendar	mapped/calendar	Map document prompt
RTF	text/rtf	Map document prompt
TXT	text/plain	Map document prompt
XML	mapped/xml	Map document prompt
PDF	application/pdf	Extract from PDF → Map document prompt
JSON	application/json	Map document prompt
XLS	application/vnd.ms-excel	Extract from XLS → Map JSON → Map document prompt
XLSX	spreadsheetml.sheet	Extract from XLSX → Map JSON → Map document prompt
IMAGE	mapped/image	HTTP Request (Medplum Binary upload)
ELSE	Any unrecognized type	Send Unsupported Response

5.6 Phase 5 — Content Extraction (Text-Based Files)

- Extract from PDF: Extracts full text content from PDF binary data.
- Extract from XLS / Extract from XLSX: Parse Excel files in both legacy and modern formats into structured JSON data.
- Map JSON: Converts extracted spreadsheet data to a stringified JSON field called data.
- Map document prompt (convergence node): Assembles the final output for all text-extractable file types, combining: extracted document text, the patient's WhatsApp caption, and the file MIME type. This structured output is returned to the parent AI Agent for LLM processing.

5.7 Phase 6 — Binary Upload Path (Images)

- HTTP Request (Medplum Binary upload): POSTs the raw binary file to <https://api.medplum.com/fhir/R4/Binary> with X-Security-Context: Patient/{patient_id} to scope the resource to the correct patient.
- Code in JavaScript: Parses the Medplum Binary response, extracting the new binary resource id and patient_id.

- HTTP Request1 (DocumentReference creation): Creates a FHIR DocumentReference resource — status: 'current', subject: Patient/{patient_id}, type: LOINC 11502-2 (Laboratory report), content attachment URL: Binary/{binary_id}.
- Code in JavaScript1: Parses the DocumentReference response, extracting document_ref_id, patient_id, and status.
- Edit Fields (Success message): Constructs the final output: "Document successfully attached to patient. Reference ID: {document_ref_id}"

5.8 Error Handling

- Send Unsupported Response: Triggered by the ELSE route. Sends the patient a WhatsApp message asking them to re-send the file as a PDF or photo.
- Send Unsupported Response1: Triggered when DocumentReference creation fails. Sends an error notification to the patient via WhatsApp.

6. Workflow 4 — Smart Patient Departure Notifier

This sub-workflow delivers real-time, personalized departure time recommendations to patients. When triggered by a patient's WhatsApp message, it simultaneously queries the clinic's live appointment queue and calculates the patient's driving time from their registered home address to the clinic — then tells the patient exactly when to leave.

6.1 Purpose

- Eliminate waiting room overcrowding by staggering patient arrivals based on live queue position.
- Reduce patient frustration by providing precise, data-driven departure recommendations.
- Leverage existing FHIR patient address data to provide accurate routing without requiring the patient to share their location manually.

6.2 Node-by-Node Breakdown

Node 1 — WhatsApp Trigger

Listens for any inbound patient WhatsApp message such as "When should I leave?" or "I'm heading to the clinic". Captures the sender's phone number for patient identification.

Node 2 — Get Access Token

Authenticates with Medplum using the OAuth2 client_credentials flow. POSTs client_id and client_secret to the Medplum token endpoint and receives a Bearer access token used for all subsequent FHIR API calls.

Node 3 — Get Doctor Speed

Fetches the last 3 completed (fulfilled) appointments from Medplum, sorted by recency. Designed to dynamically calculate the doctor's current throughput rate. The current implementation uses a fixed value of 15 minutes per patient; this node provides the data foundation for making that value fully dynamic in future iterations.

Node 4 — Search Patient

Queries Medplum's FHIR Patient resource using the patient's WhatsApp phone number (matched against the telecom field). Returns the patient's full record including their registered home address, used for geocoding.

Node 5 — Get Clinic Location

Fetches the clinic's Organization resource from Medplum using the configured Organization ID. Extracts the clinic's physical street address for geocoding.

Node 6 — Geocode Patient Address

Submits the patient's home address to the OpenRouteService geocoding API and receives GPS coordinates (longitude, latitude).

Node 7 — Geocode Clinic Address

Submits the clinic's address to the OpenRouteService geocoding API and receives GPS coordinates. After this node, the workflow has precise GPS coordinates for both the patient's origin and the clinic's destination.

Node 8 — Get Directions

Calls the OpenRouteService Directions API with the driving-car profile, using the patient's GPS coordinates as origin and the clinic's GPS coordinates as destination. Returns the full route including duration in seconds — the core travel-time data.

Node 9 — Get Appointments

Fetches up to 50 current appointments from Medplum for the clinic. Provides queue visibility — specifically the count of patients with 'arrived' status who are already at the clinic and waiting.

Node 10 — Code (JavaScript): Departure Calculation Engine

The computational core of this workflow. Combines data from both parallel tracks to produce a personalized departure recommendation:

- Travel time calculation: Converts the route duration from seconds to minutes.
- Queue wait calculation: Counts patients with 'arrived' status in the appointment list. Multiplies the count by 15 minutes (configurable per-patient service time).
- Departure calculation: $\text{leave_in} = \text{queue_wait_time} - \text{travel_time}$. Represents how many minutes from now the patient should leave home to arrive exactly when it is their turn.
- Generates a personalized message: e.g., "Hi Abrar, your drive is 22 mins. There are 3 patients ahead of you. Please leave your house in 18 minutes."
- Edge case: If $\text{leave_in} \leq 0$, the message reads: "Please leave now to reach on time!"

Node 11 — Send WhatsApp Message

Delivers the personalized departure message to the patient's WhatsApp number, closing the interaction loop with a precise, actionable recommendation.

6.3 Parallel Execution Architecture

After the initial authentication step, the workflow runs two parallel processing tracks:

- Queue Track: Get Doctor Speed → Get Appointments. Collects clinic-side queue data.
- Travel Track: Search Patient → Get Clinic Location → Geocode Patient → Geocode Clinic → Get Directions. Collects patient-side routing data.

The Get Directions output is fed into the Get Appointments node, which then flows into the JavaScript calculation engine. By the time the calculation logic executes, it has access to both travel duration and queue depth simultaneously, with no sequential dependency between the two tracks.

7. Security Architecture

7.1 Authentication

- All Medplum API calls use the OAuth2 client_credentials flow with short-lived Bearer tokens.
- WhatsApp API calls use dedicated OAuth2 credentials.
- No API keys are stored in workflow nodes — all credentials are managed through n8n's encrypted credential store.

7.2 Abuse Prevention

- Rate limiting: Redis counters enforce a 15-message-per-window threshold per phone number.
- Spam escalation: Five confirmed spam events trigger permanent blocking.
- Blocklist enforcement: The Supabase blocked_users table is checked on every inbound message, before any clinical logic executes.
- Intent classification: Non-clinical and non-greeting messages are filtered before reaching the AI agent, preventing LLM misuse.

7.3 Data Privacy

- All patient data is stored in Medplum, which provides HIPAA-eligible infrastructure with full audit logging.
- FHIR security contexts (X-Security-Context: Patient/{id}) scope all binary uploads to individual patient records.
- Vector memory entries in Supabase are filtered by patient phone number — no cross-patient memory leakage is possible.
- Redis conversation memory has a 1-hour TTL, automatically purging all conversational data after each session.

8. FHIR Resource Reference

The following FHIR R4 resources are created, read, or updated by MediReach AI across all four workflows:

FHIR Resource	Operations	Purpose
Organization	Create, Read	Clinic / healthcare provider profile
Practitioner	Create, Read	Individual doctor / clinician profile
PractitionerRole	Create, Read	Practitioner–Organization affiliation link
Patient	Create, Read, Search	Patient demographics, contact, and address
Schedule	Read	Doctor's availability schedule
Slot	Read, Patch	Individual bookable time slots
Appointment	Create, Read, Patch	Booking records with patient–practitioner references
Observation	Create, Read	Patient-reported symptoms (LOINC 11488-4)
Binary	Create	Raw file storage for patient-uploaded documents
DocumentReference	Create, Read	Links Binary files to patient records (LOINC 11502-2)

9. End-to-End Patient Journey Example

The following illustrates a complete patient interaction — from first message to real-time departure notification — showing how all four workflows collaborate:

1. Patient Ahmed sends: "I have a headache and want to see Dr. Ali tomorrow."
2. Rate limit check passes. Blocklist check passes. Message type is text — content extracted directly.
3. Intent classification returns: medical. Message routed to the AI Agent.
4. Agent calls `medplum_patient_lookup` — finds Ahmed's patient record by phone number.
5. Agent calls `get_clinic_staff` — retrieves available doctors including Dr. Ali.
6. Agent calls `get_doctor_slots` — fetches Dr. Ali's free slots for tomorrow.
7. Agent presents available slot times to Ahmed in Pakistan Standard Time (UTC+5).
8. Ahmed confirms a slot. Agent calls `book_appointment` — creates FHIR Appointment resource.
9. Agent calls `update_slot_status` — marks the slot as 'busy', preventing double-booking.
10. Agent calls `record_observation` — logs "patient reports headache" as a FHIR Observation on Ahmed's chart.
11. Booking confirmation delivered to Ahmed via WhatsApp.
12. Background memory pipeline extracts "patient reports headache" and stores it as a semantic vector in Supabase.
13. The next day, Ahmed messages: "When should I leave?" — Smart Departure Notifier calculates a 22-minute drive and 2 patients ahead, recommending Ahmed leave in 8 minutes.

10. Glossary

Term	Definition
FHIR R4	Fast Healthcare Interoperability Resources, Release 4 — the international standard for structured healthcare data exchange.
Medplum	An open-source, HIPAA-eligible FHIR R4 platform used as the clinical data store for this system.
n8n	An open-source workflow automation platform used to orchestrate all business logic and API integrations.
PractitionerRole	A FHIR resource that links a Practitioner to an Organization, expressing their employment or affiliation.
Observation	A FHIR resource for recording clinical findings, symptoms, or notes. This system uses LOINC code 11488-4.
Slot	A FHIR resource representing an available bookable time period within a healthcare schedule.
DocumentReference	A FHIR resource that points to a document or binary file and associates it with a patient record.
OAuth2 client_credentials	An authentication flow where a service authenticates using its own client ID and secret, without user involvement.
Vector Embedding	A numerical representation of text that captures semantic meaning, enabling similarity search in the Supabase memory store.
LangChain Agent	An AI agent framework that enables an LLM to autonomously select and call tools to complete a task.
TTL	Time-to-Live — the duration after which a cached record (e.g., Redis conversation history) is automatically deleted.
LOINC	Logical Observation Identifiers Names and Codes — a standardized clinical terminology system for observations and documents.
UTC+5	Pakistan Standard Time — the timezone used when presenting appointment slot times to patients.